DOCUMENTAÇÃO TÉCNICA — RPG SIMPLES

Exemplo preenchido do Modelo de Documentação Técnica

Campo	Informação
Projeto	RPG Simples
Repositório	github.com/emmanuelvrm/rpg-simples
Branch analisada	main
Snapshot de referência	commit 4843ee2c10cf33b76ad5e11d010ee76d15d7e713 (19/08/2026 UTC)
Linguagem	Python 3.10+
Interface atual	Console / terminal
Dependências externas	Nenhuma
Licença declarada	MIT
Finalidade deste documento	Referência didática de documentação técnica preenchida

Legenda de confiabilidade — IMPLEMENTADO = confirmado no repositório. INFERIDO = deduzido diretamente da estrutura/comportamento. CENÁRIO FICTÍCIO = exemplo criado exclusivamente para demonstrar implantação, banco e infraestrutura.

1. Resumo executivo

RPG Simples é um RPG de texto por turnos desenvolvido em Python, organizado em camadas. O jogador cria um herói, distribui pontos de atributo, explora três terrenos, aceita missões, combate inimigos escalados conforme seus atributos, compra itens, descansa, sobe de nível e enfrenta chefes. O projeto também contém um simulador de balanceamento que reproduz combates sem interface para medir estatisticamente taxas de vitória e duração das partidas.

Status — A aplicação principal, o simulador e a arquitetura em camadas estão IMPLEMENTADOS. Banco de dados, interface web, publicação online, redundância e failover descritos mais adiante são CENÁRIO FICTÍCIO.

1.1 Objetivo do produto

Entregar uma experiência simples de RPG por turnos e, simultaneamente, demonstrar boas práticas de organização de código Python: separação entre domínio, regras, conteúdo, interface e configuração; encapsulamento de estado em classes; validação centralizada de entrada; e balanceamento orientado por simulação.

1.2 Escopo atual

- Criação e evolução de personagem.
- Exploração de Floresta, Montanha e Caverna.
- Combate por turnos com ataque, defesa, poção e fuga.
- Missões aleatórias com chance de sucesso.
- Chefes periódicos a cada cinco missões concluídas no terreno.
- Loja, equipamentos, poções e estalagem.
- Sistema de vidas e resgate após derrota.
- Balanceamento configurável e simulador estatístico.
- Interface exclusivamente em terminal.

1.3 Fora do escopo atual

- Persistência de partidas.
- Autenticação de usuários.
- Banco de dados.
- API HTTP ou interface web.
- Multiplayer.
- Sincronização online.
- CI/CD configurado no repositório.
- Testes automatizados em diretório tests/.

2. Requisitos

2.1 Requisitos funcionais reconstruídos a partir do código

ID	Requisito	Estado	Evidência
RF-001	Permitir informar um nome não vazio para o herói.	IMPLEMENTADO	criacao.py / console.ler_texto
RF-002	Permitir distribuir 20 pontos iniciais entre Ataque e Defesa.	IMPLEMENTADO	config.PONTOS_DE_ATRIBUTO / distribuir_atributos
RF-003	Exibir status com HP, XP, Ataque, Defesa, Ouro, Poções e Vidas.	IMPLEMENTADO	Personagem.status
RF-004	Permitir explorar três terrenos: Floresta, Montanha e Caverna.	IMPLEMENTADO	jogo.montar_mapa
RF-005	Sortear encontros aleatórios durante a exploração.	IMPLEMENTADO	exploracao.explorar
RF-006	Executar combate por turnos até vitória, derrota ou fuga.	IMPLEMENTADO	combate.batalhar
RF-007	Oferecer ataque, defesa, fuga e poção quando disponível.	IMPLEMENTADO	combate._montar_opcoes
RF-008	Conceder ouro e XP ao derrotar monstros.	IMPLEMENTADO	combate._encerrar

RF-009	Permitir subida de nível com ganho de atributos.	IMPLEMENTADO	Personagem.ganhar_xp
RF-010	Oferecer missões e registrar progresso por terreno.	IMPLEMENTADO	exploracao.oferecer_missao / Terreno
RF-011	Disparar chefe a cada 5 missões concluídas.	IMPLEMENTADO	MISSOES_PARA_CHEFE
RF-012	Permitir comprar itens e poções.	IMPLEMENTADO	loja.comprar
RF-013	Permitir descansar mediante pagamento.	IMPLEMENTADO	loja.descansar
RF-014	Permitir resgate após derrota enquanto houver vidas.	IMPLEMENTADO	Personagem.ser_resgatado / jogo._resgatar
RF-015	Encerrar a aventura quando o jogador escolher sair ou perder todas as vidas.	IMPLEMENTADO	jogo.iniciar_jogo
RF-016	Permitir simular automaticamente milhares de combates para medir balanceamento.	IMPLEMENTADO	ferramentas/balanceamento.py

2.2 Requisitos não funcionais

ID	Categoria	Critério	Evidência
RNF-001	Compatibilidade	Executar em Python 3.10 ou superior.	README
RNF-002	Portabilidade	Não depender de bibliotecas externas.	README/código
RNF-003	Usabilidade	Toda escolha operacional deve ocorrer por menu numérico validado.	console.menu / ler_inteiro
RNF-004	Manutenibilidade	Parâmetros de balanceamento devem ficar centralizados.	config.py
RNF-005	Separação de responsabilidades	Entrada e saída devem ficar concentradas em ui/console.py.	arquitetura implementada
RNF-006	Testabilidade	Regras fundamentais de dano devem poder ser chamadas sem interface.	calcular_dano + simulador
RNF-007	Confiabilidade	HP não pode ficar negativo; cura não pode ultrapassar HP máximo.	models
RNF-008	Observabilidade futura	CENÁRIO FICTÍCIO: logs estruturados e healthcheck.	proposta de infraestrutura

2.3 Regras de negócio principais

Regra	Descrição
RN-001	XP necessário para subir = 40 × nível atual.
RN-002	Ao subir de nível: +12 HP máximo, +3 Ataque e +2 Defesa.
RN-003	O herói cura totalmente ao subir de nível.
RN-004	Dano mínimo de qualquer golpe = 1.
RN-005	Crítico: 22% de chance; bônus de 75% do ataque.
RN-006	Defesa reduz 60% do dano recebido no turno.
RN-007	Monstro se defende em 28% dos turnos.
RN-008	Fuga possui 50% de chance.
RN-009	Encontro ocorre em 55% das explorações.
RN-010	Missão possui 70% de chance de sucesso.
RN-011	Cada terreno possui limite padrão de 15 missões.
RN-012	Chefe surge a cada 5 missões completas.
RN-013	Jogador inicia com 3 vidas.
RN-014	Resgate remove 50% do ouro atual e restaura HP.
RN-015	Estalagem custa 6 × nível e recupera 100% do HP.
RN-016	Poção custa 30 ouro e cura 40% do HP máximo.

3. Arquitetura e organização do código

A arquitetura implementada é uma separação em camadas simples. A dependência predominante flui da orquestração e regras para o domínio e conteúdo; a interface é utilizada pelos módulos de aplicação, enquanto os parâmetros comuns residem em config.py. O projeto deliberadamente evita frameworks e dependências externas.

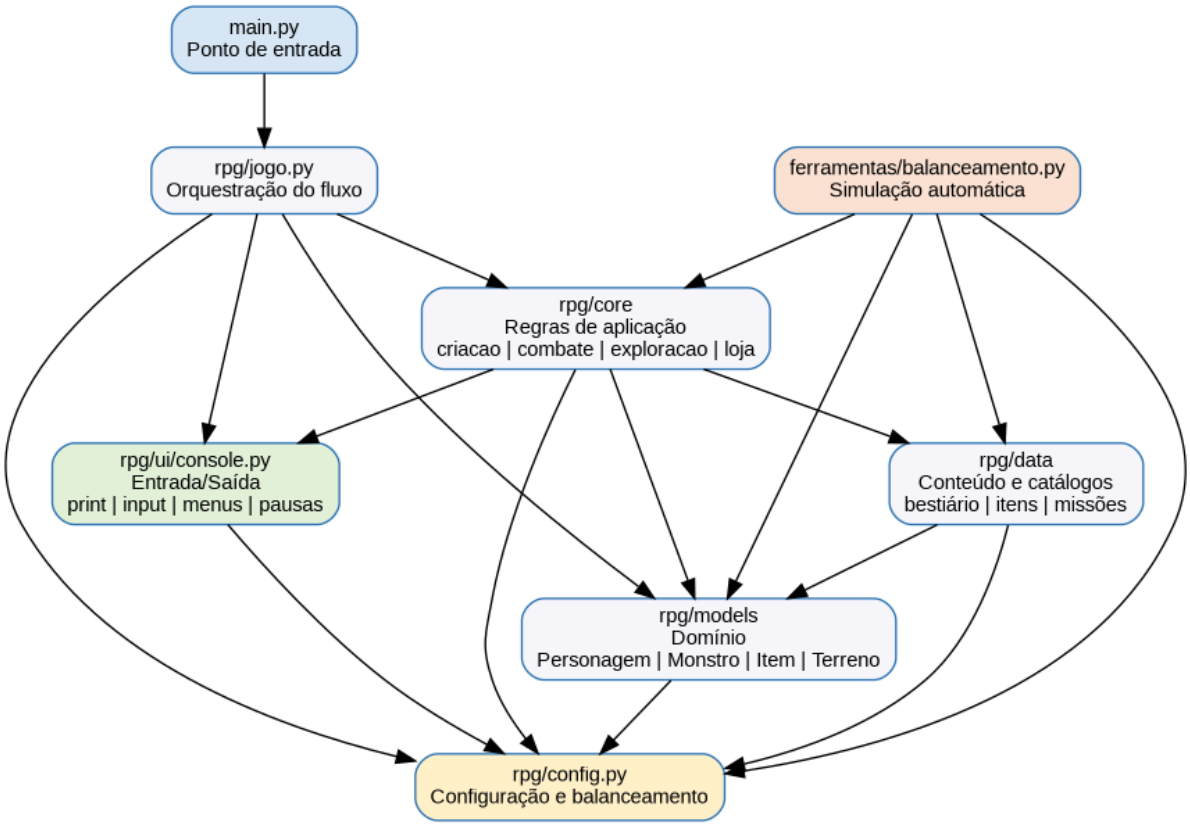


Figura 1 — Diagrama de componentes e dependências do código atual.

3.1 Estrutura de diretórios

```
rpg-simples/
├── main.py
├── ferramentas/
│   └── balanceamento.py
└── rpg/
    ├── config.py
    ├── jogo.py
    ├── models/
    │   ├── personagem.py
    │   ├── monstro.py
    │   ├── item.py
    │   └── terreno.py
    ├── data/
    │   ├── bestiario.py
    │   ├── itens.py
    │   └── missoes.py
    ├── core/
    │   ├── criacao.py
    │   ├── combate.py
    │   ├── exploracao.py
    │   └── loja.py
    └── ui/
        └── console.py
```

3.2 Responsabilidades por camada

Camada	Responsabilidade	Exemplos
Entrada	Inicializar aplicação e tratar interrupção de teclado.	main.py
Orquestração	Manter loop principal, mapa, resgate e fim de jogo.	rpg/jogo.py
Domínio	Guardar estado e comportamento intrínseco das entidades.	Personagem, Monstro, Item, Terreno
Dados	Catálogos e textos editáveis.	ARQUETIPOS, CATALOGO, MISSOES
Regras	Executar casos de uso e regras de aplicação.	criação, combate, exploração, loja
Interface	Centralizar print/input, menus, validação e pausas.	ui/console.py
Configuração	Centralizar todos os parâmetros de balanceamento.	config.py
Ferramentas	Executar simulações sem interface.	balanceamento.py

4. Modelagem orientada a objetos

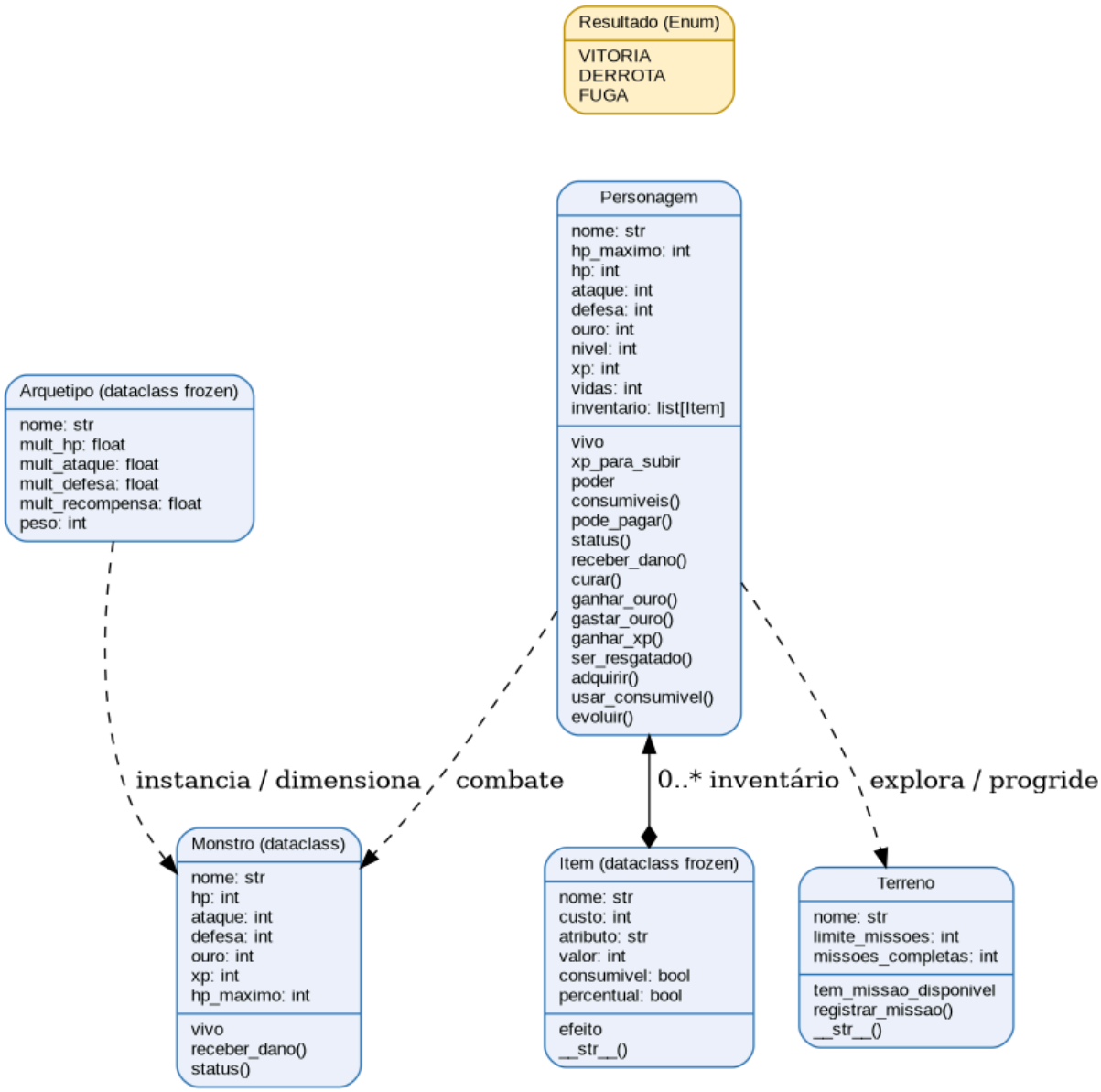


Figura 2 — Diagrama de classes reconstruído do código atual.

4.1 Classe Personagem

Entidade central do domínio. Mantém identidade, atributos de combate, progressão, economia, vidas e inventário. O próprio objeto garante invariantes importantes: HP não negativo, cura limitada ao máximo, gasto condicionado a saldo e consumo efetivo de itens.

Grupo	Membros
Estado	nome, hp_maximo, hp, ataque, defesa, ouro, nivel, xp, vidas, inventario
Consultas	vivo, xp_para_subir, poder, consumiveis(), pode_pagar(), status()
Combate/vida	receber_dano(), curar(), ser_resgatado()
Economia	ganhar_ouro(), gastar_ouro(), adquirir()
Progressão	ganhar_xp(), evoluir()
Inventário	usar_consumivel(), _aplicar_efeito()

4.2 Monstro, Item e Terreno

Classe	Papel	Características
Monstro	Representar um inimigo concreto.	dataclass; HP, ataque, defesa, ouro, XP; recebe dano e expõe status.
Item	Representar um item comprável.	dataclass imutável; custo, atributo, valor; pode ser consumível e percentual.
Terreno	Representar local explorável e progresso de missões.	Nome, limite e contador de missões; informa disponibilidade.

4.3 Arquétipo e geração de inimigos

Arquetipo não representa o monstro em batalha; representa multiplicadores usados para construir uma instância de Monstro adequada ao estado atual do jogador. Isso evita inimigos fixos que se tornam irrelevantes em níveis altos.

5. Fluxos de negócio

5.1 Fluxo geral de exploração

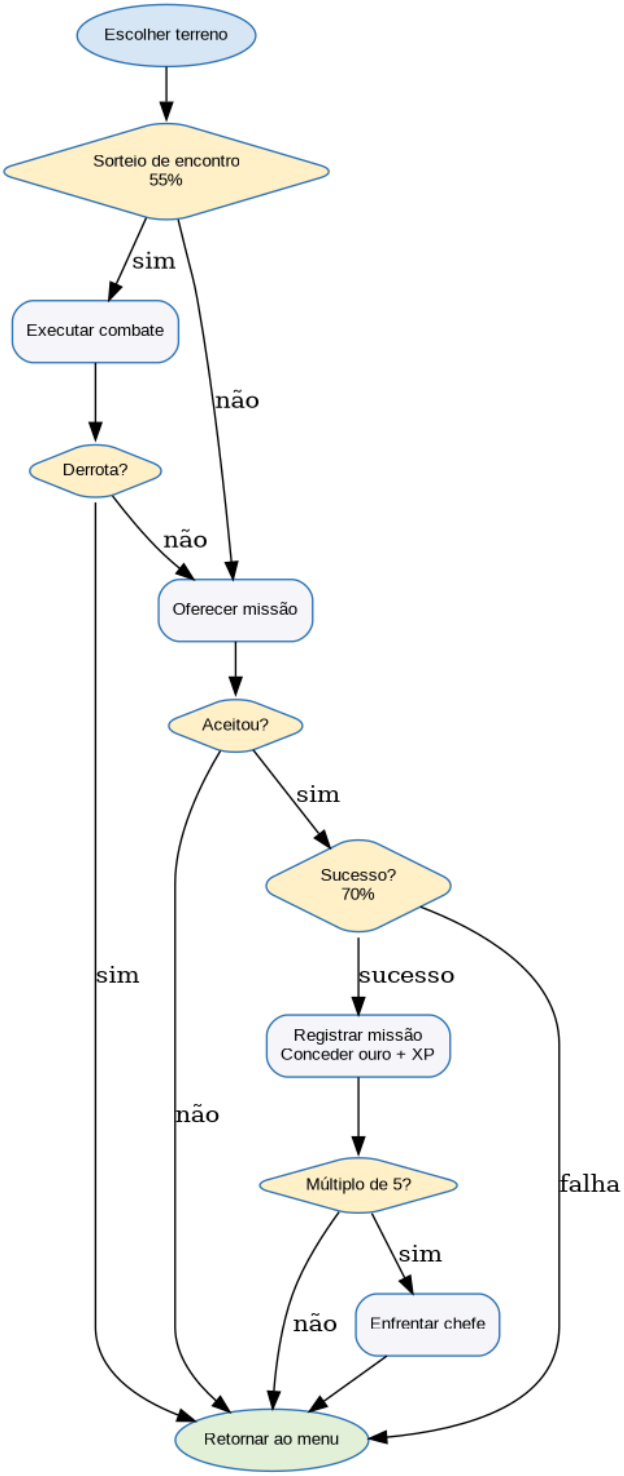


Figura 3 — Fluxo de uma ação de exploração.

5.2 Sequência simplificada do combate

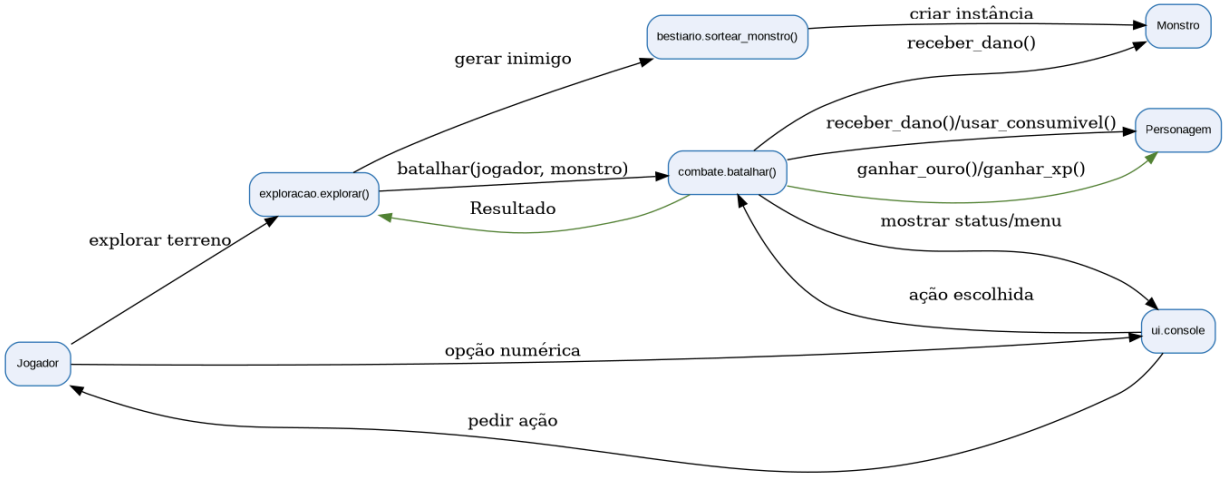


Figura 4 — Sequência simplificada entre exploração, bestiário, combate, UI e entidades.

5.3 Estados de encerramento do combate

Resultado	Condição	Efeito posterior
VITORIA	HP do monstro chega a 0.	Concede ouro e XP; pode ocorrer level up.
DERROTA	HP do personagem chega a 0.	Fluxo principal tenta resgate; se não houver vidas, fim de jogo.
FUGA	Jogador escolhe fugir e sorteio < 50%.	Combate encerra sem recompensa.

6. Balanceamento e parâmetros

O projeto adota balanceamento orientado a turnos. O inimigo é dimensionado para sobreviver aproximadamente TURNOS_PARA_VENCER golpes e derrubar o herói em aproximadamente TURNOS_PARA_PERDER golpes, considerando multiplicadores do arquétipo. Isso produz dificuldade relativa ao estado atual do jogador.

Parâmetro	Valor	Efeito
HP_INICIAL	50	HP máximo inicial.
PONTOS_DE_ATRIBUTO	20	Distribuição inicial.
XP_PARA_SUBIR	40 × nível	Curva de progressão.
TURNOS_PARA_VENCER	6	Durabilidade média do inimigo.
TURNOS_PARA_PERDER	5	Letalidade média do inimigo.
CHANCE_CRITICO	22%	Vantagem ofensiva do herói.
BONUS_CRITICO	75% do ataque	Dano extra crítico.
REDUCAO_AO_DEFENDER	60%	Mitigação quando defendendo.
CHANCE_MONSTRO_DEFENDER	28%	Turnos sem ataque do inimigo.
CHANCE_DE_FUGA	50%	Probabilidade de fuga.
CHANCE_DE_ENCONTRO	55%	Probabilidade de batalha ao explorar.
CHANCE_SUCESSO_MISSAO	70%	Probabilidade de missão concluída.
VIDAS_INICIAIS	3	Número de derrotas resgatáveis.
PERDA_DE_OURO_NA_DERROTA	50%	Penalidade do resgate.

6.1 Evidência de simulação documentada no projeto

Build inicial	Nv 1	Nv 3	Nv 6	Nv 10	Nv 15
Equilibrado 12/8	78,7%	81,0%	81,5%	80,4%	80,2%
Ofensivo 18/2	80,9%	78,7%	81,7%	81,1%	79,2%
Defensivo 4/16	79,3%	78,6%	79,2%	82,3%	82,5%
Espalhado 10/10	78,3%	82,1%	79,6%	81,1%	81,3%

Observação — As taxas acima são declaradas no README e correspondem à execução de 3.000 combates por célula. O simulador utiliza random.seed(4242) no ponto de entrada para favorecer reprodutibilidade.

7. Interface e experiência de uso

A UI atual é textual. O módulo console.py é o único local destinado a chamadas diretas de print() e input(). Essa decisão reduz acoplamento e torna plausível substituir a apresentação futuramente por outra interface, desde que seja criado um adaptador equivalente.

Função	Responsabilidade
exibir	Saída simples sem pausa.
narrar	Saída acompanhada de atraso narrativo.
titulo	Cabeçalho visual centralizado.
separador	Separação visual horizontal.
aguardar	Controla time.sleep conforme config.
pausa	Aguarda ENTER ao fim de uma cena.
ler_texto	Valida texto não vazio.
ler_inteiro	Valida inteiro e intervalo.
menu	Renderiza opções e devolve índice em base 1.
confirmar	Converte confirmação em menu numérico.

8. Dados e conteúdo

8.1 Bestiário

Arquétipo	HP	Ataque	Defesa	Recompensa	Peso
Lobo	0,70	0,95	0,70	0,80	16
Goblin	0,80	0,90	0,80	0,85	16
Orc	1,05	1,05	1,00	1,10	12
Troll	1,40	1,00	0,80	1,25	9
Mago sombrio	0,70	1,30	0,55	1,25	8
Elfo negro	1,00	1,15	1,15	1,45	6
Dragão	1,55	1,20	1,15	2,00	3
Dragão brilhante (chefe)	1,41	0,95	1,10	4,00	especial

8.2 Catálogo da loja

Item	Custo	Atributo	Valor	Consumível	Percentual
Poção de Cura	30	hp	40	Sim	Sim
Espada	50	ataque	6	Não	Não
Escudo	40	defesa	5	Não	Não
Armadura	80	defesa	10	Não	Não

8.3 Missões

O catálogo contém 11 textos de missão. O mecanismo atual seleciona uma descrição aleatória; a missão não possui tipo, requisitos ou recompensas específicas. A diferença entre missões é puramente narrativa.

9. Código-fonte e Git

Item	Situação
Repositório	emmanuelvrm/rpg-simples
Branch padrão	main
Commit de referência	4843ee2c10cf33b76ad5e11d010ee76d15d7e713
Último commit recuperado	Update README.md — 19/08/2026 20:34 UTC
Estratégia de branches	Não documentada no repositório.
CI/CD	Não identificado.
Arquivo de dependências	Não necessário na versão atual; somente stdlib.
Testes automatizados	Diretório tests/ não identificado.

9.1 Execução local

```
git clone https://github.com/emmanuelvrm/rpg-simples.git
cd rpg-simples
python main.py
```

9.2 Execução do simulador

```
python ferramentas/balanceamento.py
```

10. Estratégia de testes

Estado atual — O repositório contém um simulador de balanceamento, mas não foi identificado um conjunto formal de testes unitários/integração em tests/. As tabelas a seguir são uma proposta de documentação e de cobertura recomendada.

10.1 Casos de teste recomendados

ID	Unidade	Cenário	Resultado esperado
T-001	Personagem.curar	HP=40/50; curar 20	HP final=50; retorno=10
T-002	Personagem.receber_dano	HP=10; dano=15	HP final=0
T-003	gastar_ouro	ouro=20; custo=30	False; ouro permanece 20
T-004	ganhar_xp	nivel=1; ganhar 40	nivel=2; atributos acrescidos; XP=0
T-005	ser_resgatado	vidas=3; ouro=100; HP=0	vidas=2; ouro=50; HP cheio
T-006	calcular_dano	ataque menor que defesa	resultado mínimo=1
T-007	calcular_dano crítico	critico=True	dano inclui 75% do ataque
T-008	Terreno.registrar_missao	0 missões	contador=1
T-009	descansar	HP cheio	False; nenhum ouro gasto
T-010	comprar	ouro insuficiente	False; inventário inalterado
T-011	_construir monstro	diferentes níveis/builds	estatísticas escalam sem valores inválidos
T-012	console.ler_inteiro	entrada não numérica/fora da faixa	repete até valor válido

10.2 Testes estatísticos

- Executar ao menos 3.000 simulações por build e nível quando alterar config.py ou bestiario.py.
- Comparar taxa de vitória de builds ofensiva, defensiva, equilibrada e 10/10.
- Validar duração média de combate.
- Medir chefe com 0, 1, 2 e 3 poções.
- Definir faixa aceitável antes da alteração; exemplo didático: 75% a 85% para monstros comuns.

11. Segurança e riscos do código atual

Risco / limitação	Impacto	Situação / mitigação
Entrada de usuário	Baixo	Validada e limitada a texto/números; não há execução de comandos.
Random não criptográfico	Nenhum para o propósito do jogo	random é apropriado para gameplay.
Sem persistência	Perda total do progresso ao encerrar.	Limitação funcional atual.
Sem testes formais	Regressões podem passar despercebidas.	Adicionar pytest.
__pycache__ versionado	Ruído no repositório.	Adicionar .gitignore e remover caches rastreados.
UI e core ainda acoplados	core importa console diretamente.	Para UI intercambiável real, introduzir porta/interface de apresentação.
Funções privadas testadas indiretamente	Pode dificultar cobertura fina.	Testar API pública e extrair serviços puros quando necessário.

12. Banco de dados — cenário fictício de evolução

CENÁRIO FICTÍCIO — O RPG atual não usa banco de dados. Esta seção demonstra como documentar uma evolução que adicionasse salvamento de partidas e ranking sem afirmar que ela existe no repositório.

Proposta: PostgreSQL para armazenar usuários, personagens, progresso por terreno, inventário e histórico resumido de partidas. A lógica de combate e domínio permaneceria em Python; a persistência seria adicionada por uma camada repository para evitar dependência direta do domínio com SQL.

Tabela	Campos principais	Relacionamentos
usuario	id, nome, email, senha_hash, criado_em	1:N personagem
personagem	id, usuario_id, nome, hp, hp_maximo, ataque, defesa, ouro, nivel, xp, vidas	N:1 usuario; 1:N inventario; 1:N progresso_terreno
inventario	id, personagem_id, item_codigo, quantidade	N:1 personagem
progresso_terreno	id, personagem_id, terreno, missoes_completas	N:1 personagem
partida	id, personagem_id, inicio, fim, nivel_final, combates_vencidos, motivo_fim	N:1 personagem

12.1 Estratégia de persistência proposta

- Repository Pattern entre core/models e infraestrutura de dados.
- Migrações versionadas.
- Transações na compra de itens e salvamento de progresso.
- Backup lógico diário com pg_dump.
- Credenciais apenas em variáveis de ambiente.

13. Publicação online — cenário fictício

CENÁRIO FICTÍCIO — Para tornar o exemplo compatível com a etapa de publicação online, considera-se um adaptador Flask que reutiliza as regras do core. Esse adaptador não existe na versão analisada.

A interface web fictícia utilizaria Flask apenas como camada de apresentação/controlador HTTP. O domínio Personagem/Monstro/Item/Terreno e serviços de combate/exploração seriam reaproveitados após pequena refatoração para remover chamadas diretas ao console dentro de core.

Ambiente	Endereço fictício	Objetivo
Desenvolvimento	localhost:5000	Desenvolvimento e testes.
Homologação	rpg-hml.exemplo.local	Validação antes da produção.
Produção	rpg.exemplo.local	Acesso público fictício.

14. Infraestrutura — cenário fictício completo

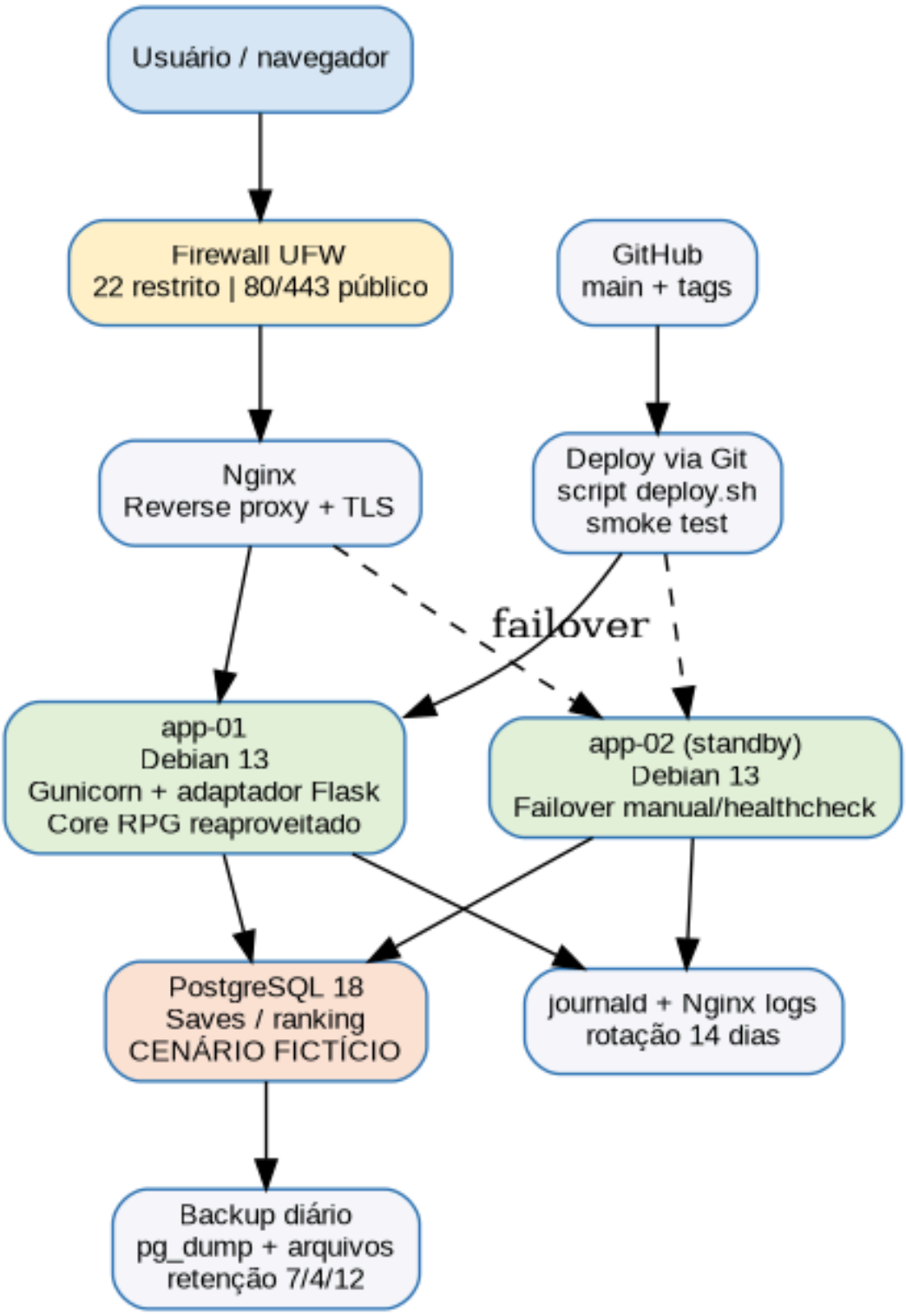


Figura 5 — Topologia fictícia para fins didáticos. Não representa recursos reais do repositório.

14.1 Inventário fictício

Recurso	Configuração	Função
app-01	VM, 2 vCPU, 2 GB RAM, 20 GB SSD, Debian 13	Aplicação primária.
app-02	VM, 2 vCPU, 2 GB RAM, 20 GB SSD, Debian 13	Standby para failover.
db-01	VM, 2 vCPU, 4 GB RAM, 40 GB SSD, Debian 13 + PostgreSQL 18	Persistência fictícia.
backup-01	Storage separado, 100 GB	Backups e retenção.
Nginx	No app-01/app-02 ou proxy dedicado	TLS e reverse proxy.

14.2 Rede fictícia

Item	Valor fictício
Rede de servidores	10.20.30.0/24
app-01	10.20.30.11
app-02	10.20.30.12
db-01	10.20.30.21
backup-01	10.20.30.31
Gateway	10.20.30.1
DNS interno	10.20.30.2
SSH	Permitido apenas da rede administrativa 10.20.10.0/24
HTTP/HTTPS	80/443 públicos no proxy
PostgreSQL	5432 somente dos hosts app-01/app-02

14.3 Usuários e grupos fictícios

Identidade	Tipo	Finalidade	Permissões
rpgapp	usuário de sistema	Executar Gunicorn/aplicação.	Sem sudo; acesso somente /opt/rpg-simples e logs necessários.
deploy	usuário humano/técnico	Atualizar aplicação.	sudo restrito para systemctl do serviço e leitura de logs.
rpgadmin	grupo	Administradores do serviço.	SSH e operações autorizadas.
postgres	usuário de sistema	Banco de dados.	Gerenciado pelo pacote PostgreSQL.

14.4 Estrutura fictícia no servidor

```
/opt/rpg-simples/
├── app/              # clone/release atual
├── venv/             # ambiente virtual
├── releases/         # versões anteriores
└── shared/
    └── .env           # permissões 600, fora do Git

/var/log/nginx/
/var/backups/rpg-simples/
```

15. Systemd e processos — cenário fictício

O serviço web fictício poderia ser gerenciado pelo systemd. Para o RPG puramente de terminal, a execução continua manual com python main.py; portanto o exemplo abaixo pertence exclusivamente à evolução web.

```
[Unit]
Description=RPG Simples Web
After=network.target

[Service]
User=rpgapp
Group=rpgapp
WorkingDirectory=/opt/rpg-simples/app
EnvironmentFile=/opt/rpg-simples/shared/.env
ExecStart=/opt/rpg-simples/venv/bin/gunicorn -w 2 -b 127.0.0.1:8000 webapp:app
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
```

15.1 Operações

Ação	Comando
Atualizar units	sudo systemctl daemon-reload
Habilitar	sudo systemctl enable rpg-simples
Iniciar	sudo systemctl start rpg-simples
Reiniciar	sudo systemctl restart rpg-simples
Status	systemctl status rpg-simples
Logs	journalctl -u rpg-simples -f

16. Firewall e segurança — cenário fictício

Regra	Origem	Destino	Ação
FW-001	Internet	80/tcp	ALLOW — redirecionar para HTTPS
FW-002	Internet	443/tcp	ALLOW
FW-003	10.20.10.0/24	22/tcp	ALLOW administração
FW-004	qualquer outro	22/tcp	DENY
FW-005	10.20.30.11-12	db-01:5432	ALLOW
FW-006	demais origens	db-01:5432	DENY

- TLS obrigatório no ambiente de produção fictício.
- Segredos em .env com chmod 600 e fora do Git.
- Usuário do serviço sem shell administrativo e sem sudo amplo.
- Atualizações de segurança periódicas.
- Nginx com limites básicos de requisição e cabeçalhos de segurança.

17. Logs, diagnóstico e monitoramento — cenário fictício

Fonte	Local/comando	Conteúdo
Aplicação	journalctl -u rpg-simples	Inicialização, erros, exceções, eventos operacionais.
Nginx access	/var/log/nginx/access.log	Requisições HTTP.
Nginx error	/var/log/nginx/error.log	Erros de proxy/TLS.
Sistema	journalctl / dmesg	Serviços, recursos e kernel.
PostgreSQL	journald ou log_directory	Conexões e erros do banco.

17.1 Runbook de diagnóstico

Sintoma	Verificação	Ação
Site indisponível	systemctl status; ss -lntp; curl localhost:8000	Reiniciar serviço se necessário e consultar journal.
502 Bad Gateway	Nginx error.log; processo Gunicorn	Corrigir serviço upstream.
Erro de banco	pg_isready; journal PostgreSQL; conectividade 5432	Restaurar serviço/conexão; validar credenciais.
Disco cheio	df -h; du -sh /var/log/*	Rotacionar logs/limpar arquivos conforme política.
Uso de memória alto	free -h; ps aux --sort=-%mem	Investigar worker, reduzir workers ou ampliar recurso.

18. Backup, recuperação e failover — cenário fictício

18.1 Política de backup

Objeto	Frequência	Retenção	Destino
PostgreSQL — pg_dump	Diária 02:00	7 diários + 4 semanais + 12 mensais	backup-01
Configuração Nginx/systemd	Após alteração + semanal	4 semanais	backup-01
Arquivos shared	Diária	7 dias	backup-01
Código	Versionado continuamente	Histórico Git	GitHub + clone local

18.2 Recuperação

- Provisionar VM limpa com Debian 13.
- Instalar Git, Python, Nginx e dependências.
- Clonar tag estável do GitHub.
- Restaurar .env e configurações.
- Restaurar dump do PostgreSQL.
- Habilitar e iniciar serviços.
- Executar smoke tests.
- Liberar tráfego somente após validação.

18.3 Failover fictício

Em falha do app-01, o proxy/DNS interno é alterado para app-02. O app-02 usa o mesmo banco db-01. Esse arranjo protege apenas contra falha do nó de aplicação; o banco continua sendo ponto único de falha. Em uma arquitetura mais madura, PostgreSQL teria réplica e mecanismo de promoção controlada.

19. Deploy pelo Git — cenário fictício

```
#!/usr/bin/env bash
set -euo pipefail
cd /opt/rpg-simples/app
git fetch --tags origin
git checkout "$1"
/opt/rpg-simples/venv/bin/pip install -r requirements.txt
```

```
sudo systemctl restart rpg-simples
curl --fail http://127.0.0.1:8000/health
```

O script acima é apenas ilustrativo. Em produção real, seria preferível implantar releases em diretórios versionados e trocar um symlink current, reduzindo o risco de estado parcial e facilitando rollback.

19.1 Rollback fictício

- Identificar última tag estável.
- Trocar release/current para a versão anterior ou checkout da tag anterior.
- Reverter migração apenas quando explicitamente suportada.
- Reiniciar serviço.
- Executar smoke test e consultar logs.

20. Rastreabilidade

Requisito	Componente principal	Teste recomendado	Evidência
RF-002	core/criacao.py + Personagem	T distribuição 20 pontos	Código atual
RF-005	core/exploracao.py	Teste com random mockado	Código atual
RF-006	core/combate.py	Testes de fluxo de Resultado	Código atual
RF-008	combate._encerrar + Personagem	Teste de recompensa	Código atual
RF-009	Personagem.ganhar_xp	T-004	Código atual
RF-011	exploracao.oferecer_missao	Teste contador múltiplo de 5	Código atual
RF-012	core/loja.py	T-010	Código atual
RF-014	Personagem.ser_resgatado	T-005	Código atual
RF-016	ferramentas/balanceamento.py	Execução com seed	Simulador atual

21. Decisões arquiteturais (ADR) reconstruídas

ADR	Decisão	Justificativa observável	Trade-off
ADR-001	Python puro / zero dependências	Facilidade de execução e foco didático.	Menos recursos prontos de UI/persistência.
ADR-002	Centralizar configuração em config.py	Ajustar dificuldade sem alterar regras.	Arquivo pode crescer se o projeto expandir.
ADR-003	Centralizar I/O em ui/console.py	Reduz espalhamento de print/input.	Core ainda importa a UI diretamente.
ADR-004	Inimigos escalados por atributos do herói	Mantém relevância em múltiplos níveis.	Balanceamento fica mais abstrato.
ADR-005	Simulação automatizada de balanceamento	Evita ajuste somente por tentativa e erro.	Não substitui testes de regras nem playtest humano.
ADR-006	Poção percentual	Permanece útil em níveis altos.	Cura absoluta menos previsível.

22. Riscos, limitações e dívida técnica

Item	Tipo	Prioridade sugerida	Tratamento
Ausência de testes automatizados formais	Qualidade	Alta	Criar pytest para models/core.
__pycache__ presente no repositório	Higiene Git	Média	Adicionar .gitignore e remover artefatos.
core depende diretamente de console	Arquitetura	Média	Introduzir interface/porta de UI ou eventos de domínio.
Sem persistência	Produto	Conforme objetivo	Adicionar repository somente se salvar partidas for requisito.
Missões apenas textuais	Modelagem	Baixa	Criar entidade Missao se houver regras específicas.
Sem type checking/lint documentado	Qualidade	Baixa	Adicionar ruff/mypy opcionalmente.
Sem pipeline CI	DevOps	Média	Executar testes e simulador reduzido em push/PR.

23. Roadmap técnico sugerido

Prioridade	Evolução	Resultado esperado
P1	Adicionar .gitignore e suíte pytest	Base de qualidade e repositório limpo.
P1	Refatorar core para retornar eventos/mensagens em vez de chamar console	Facilitar UI web/desktop e testes.
P2	Introduzir save/load em JSON local antes de banco	Persistência simples com baixo custo.
P2	Criar entidade Missao se missões ganharem regras próprias	Domínio mais expressivo.
P3	Criar adaptador Flask ou Tkinter	Nova UI reutilizando regras.
P3	Persistência PostgreSQL e autenticação somente se houver requisito online	Escalar para publicação real.
P4	CI/CD, monitoramento e redundância	Operação mais madura.

24. Checklist de reprodução

- Python 3.10+ instalado.
- Clonar o repositório.
- Executar python main.py.
- Criar personagem e distribuir todos os 20 pontos.
- Explorar ao menos um terreno.
- Executar combate e testar defesa/fuga.
- Obter ouro e visitar a cidade.
- Comprar item ou descansar.
- Executar python ferramentas/balanceamento.py.
- Comparar resultados com o README.

25. Referências

- Repositório analisado: https://github.com/emmanuelvrm/rpg-simples
- README.md, main.py, rpg/config.py, rpg/jogo.py, rpg/models/*, rpg/core/*, rpg/data/*, rpg/ui/console.py e ferramentas/balanceamento.py.
- Snapshot utilizado: branch main; commit mais recente recuperado 4843ee2c10cf33b76ad5e11d010ee76d15d7e713, de 19/08/2026 20:34 UTC.

Nota final — Este documento foi intencionalmente preenchido em nível superior ao que normalmente seria necessário para um projeto deste porte. O objetivo é servir de referência aos alunos sobre como evidenciar requisitos, modelagem, código, testes, decisões e operação sem transformar cada etapa do projeto em um documento isolado.